
LLD – DAY 8

Design Patterns (Strategy Design Pattern)

◇ What is a Design Pattern & Why Do We Use It?

☑ Definition

Design Pattern ek **proven solution** hota hai jo **commonly occurring software design problems** ko solve karta hai.

🗨 Hinglish Explanation

- Jab hum **nayi application develop** karte hain
- To **same type ke problems** baar-baar aate hain
- Ye problems **pehle bhi bahut log face kar chuke hote hain**
- Unhone jo **best solution nikala**, wahi **Design Pattern** kehlata hai

📌 Important Line

Change is the only constant (even in real life)

◇ Design Patterns kya suggest karte hain?

Application ke **2 parts** hote hain:

1. **Static Part** → Jo rarely change hota hai
2. **Dynamic Part** → Jo baar-baar change hota hai

🎯 Core Idea

Dynamic part ko static part se isolate karo

📌 Isi idea par **Strategy Pattern** based hai.

PROBLEM: Inheritance-Based Robot Design

🗨 Problem Statement

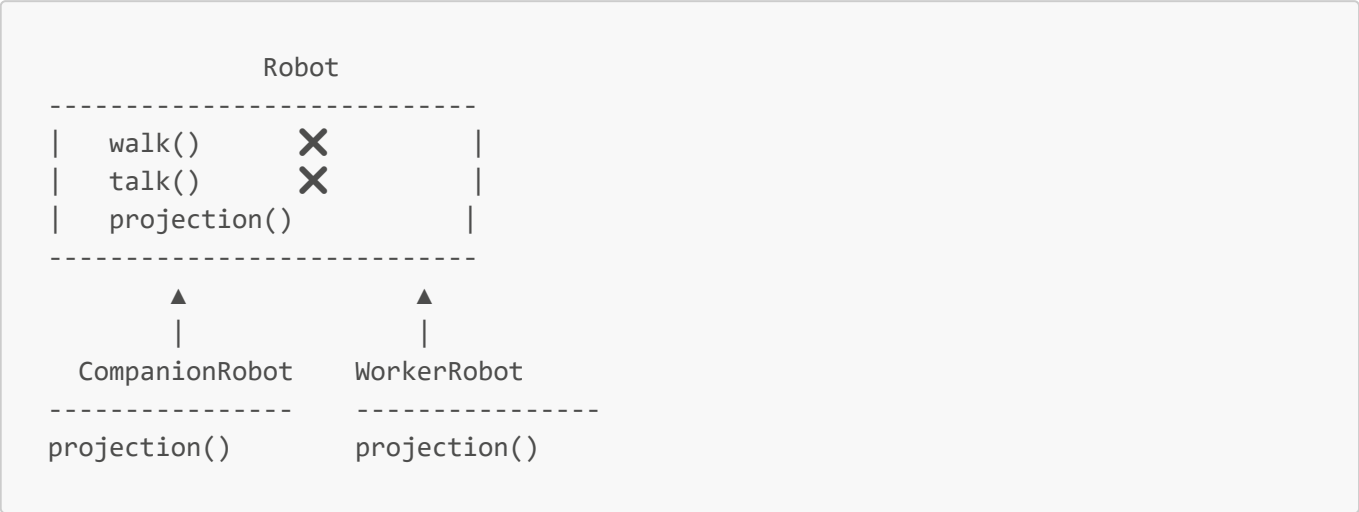
Har robot ke paas common features hain:

- `walk()`
- `talk()`
- `projection()`

Isliye humne ek **base class Robot** banayi aur uske child classes banaye:

- CompanionRobot
- WorkerRobot

 TEXT UML DIAGRAM – INHERITANCE APPROACH (PROBLEM)



✗ Problem with Inheritance

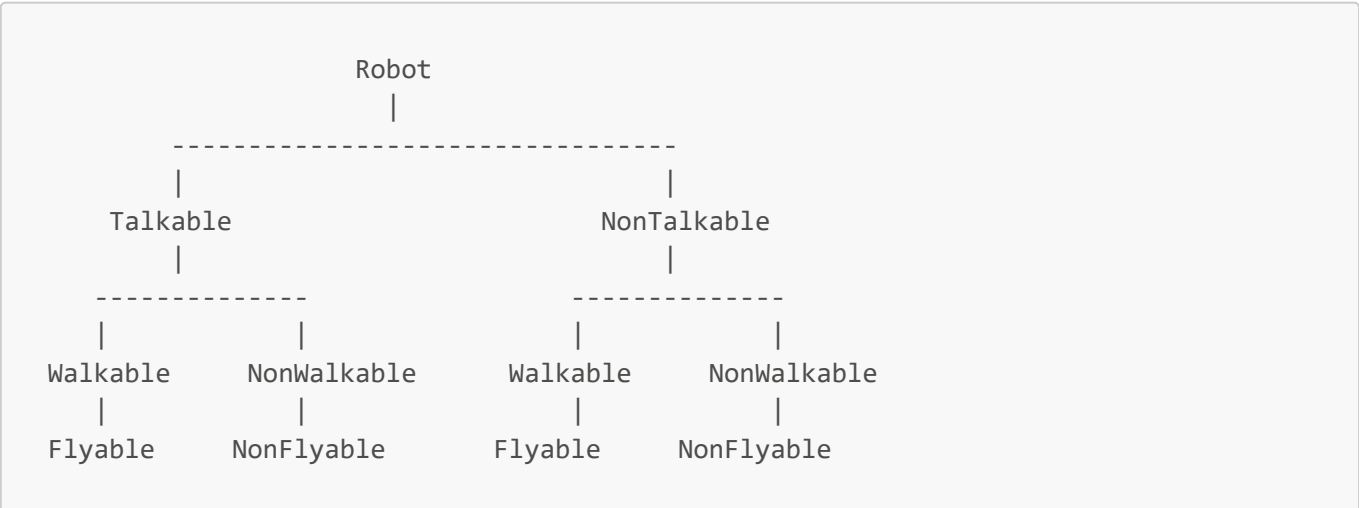
Scenario:

- Aaj robot **walk + talk** karta hai
- Kal new feature aaya → **fly()**

✗ Issues:

1. Har robot me **fly()** ka code repeat karna padega → **✗ DRY violation**
2. Agar kisi robot ko fly nahi karna → empty / dummy method likhna padega
3. Multiple combinations create ho jaate hain 🖱️

 Inheritance Explosion Problem



Result

- Confusing hierarchy
- Maintenance nightmare
- OCP break hota hai

Key Observation

Inheritance is not the solution to every problem

Problems with Inheritance:

- Code reuse properly nahi hota
- New feature add karne ke liye **existing code modify**
- **✗ OCP break**

Solution: Favor Composition Over Inheritance

Composition > Inheritance

Aur isi solution ka naam hai  **Strategy Design Pattern**

Strategy Pattern

Define a family of algorithms, put them into separate classes, and make them interchangeable at runtime

Key Idea

Behavior ko class ke andar define mat karo, usko alag object bana do aur inject karo.

 This is called **Strategy Pattern**

TEXT UML DIAGRAM – SOLUTION (STRATEGY PATTERN)

◇ Robot (Context Class)

```
Robot
```

```
-----  
Talkable  t;  
Walkable  w;  
Flyable   f;  
-----
```

```
walk()      -> w.walk()
talk()      -> t.talk()
fly()       -> f.fly()
projection()
```

🔗 Robot ab **behavior implement nahi karta**, sirf **delegate karta hai** ✓

◇ Strategy Interfaces

```
<<interface>> Talkable
-----
talk()

<<interface>> Walkable
-----
walk()

<<interface>> Flyable
-----
fly()
```

◇ Concrete Strategy Classes

🗣️ Talk Strategies

NormalTalk	NoTalk
-----	-----
talk()	talk()

🚶 Walk Strategies

NormalWalk	NoWalk
-----	-----
walk()	walk()

✈️ Fly Strategies

NormalFly	NoFly
-----	-----
fly()	fly()

🔗 Ye sab **Concrete Strategies** hain

◇ Concrete Robot

```
CompanionRobot
-----
projection()
```

🔗 Projection robot-specific hai isliye inheritance sirf **static behavior** ke liye use hua ✓

◇ Runtime Composition (IMPORTANT)

```
Robot* robot = new CompanionRobot(
    new NormalTalk(),
    new NormalWalk(),
    new NormalFly()
);
```

🔗 Ab:

- Fly behavior change karna ho → sirf **Flyable class change**
- Robot class untouched ✓

◇ Optimized Benefit

Agar kal naya flying behavior aaye 🔄 sirf **new Flyable class add karo**

✗ Existing code modify nahi ✓ **OCP follow**

💧 WHY THIS SOLUTION IS PERFECT

✓ SRP

- Robot → coordination
- Talkable / Walkable / Flyable → behavior

✓ OCP

- New behavior = new class
- Old code untouched

✓ DRY = Don't Repeat Yourself

- No duplicate code

✓ Composition > Inheritance

◇ Standard UML – Strategy Pattern



📄 Method Flow (VERY IMPORTANT)

```

Client.execute()
{
    s->run();
}
  
```

🔗 Matlab:

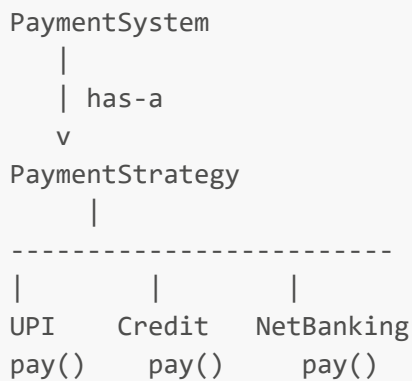
Client directly algorithm nahi chalata

Client Strategy interface ko call karta hai

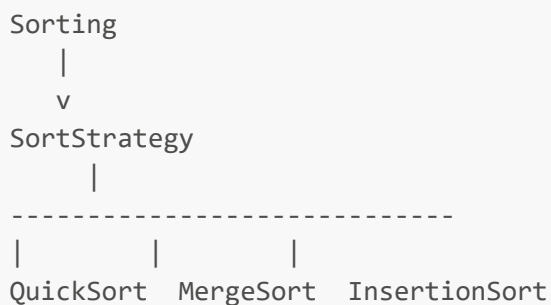
Actual behavior ConcreteStrategy decide karti hai

◇ Real Life Examples

1 Payment System



2 DSA Example – Sorting



◇ Conclusion

- 1 Encapsulate what varies & separate from what remains same
 - 2 Solution to inheritance problem is NOT more inheritance
 - 3 Favor Composition over Inheritance
 - 4 Code to interface, not to concrete class
 - 5 DRY – Do Not Repeat Yourself
-

🎯 One-Line Interview Answer

Strategy Pattern allows behavior to be selected at runtime using composition instead of inheritance.
